```
      // Normalize spaces
      pageContent = pageContent.replace(/\s+/g, ' ');
      pageContent = pageContent.replace(/(\s*l\s*){2,}/g, ' ');

      // Filter out non-content lines
      const filteredDocument = pageContent.split('\n').filter(line =>
        !line.includes('References') && !line.includes('Acknowledgments')
      )[0];
      doc.pageContent = filteredDocument;
      return doc;
    }

    return null;
  }).filter(doc => doc.content !== null);
// Filter out any null entries from the result
}
```

Removes empty spaces and irregular characters

Filters out any lines that contain the words "References" or "Acknowledgments," as these are likely not part of the main content

Measuring the cost of multiple LLM calls

One of the key challenges when using LLMs in an application like the summarization tool is the cost management aspect, particularly when dealing with large documents.

The problem arises because LLMs have a limited context window; they can only process a certain amount of text in a single API call. When the input document exceeds this context window, we are forced to make multiple calls to the LLM to process the full content. Each of these API calls incurs a cost, which can quickly ramp up, especially for applications that need to process large volumes of text.

To help manage these costs, LangChain.js provides a built-in mechanism for tracking token usage during LLM calls using callback functions. This allows you to gain insights about the number of tokens used on each call. The following listing shows an example of how you can set up this token usage tracking.

Listing 7.4 Tracking usage of tokens

```
import TokenTracker from "./TokenTracker.js";
const tokenTracker = new TokenTracker();
const model = new ChatGoogleGenerativeAI({
  apiKey: apiKey,
  model: "gemini-2.0-flash",
  streaming: false,
  callbacks: [
    {
      handleLLMEnd: (output) => {
        const { message } = output.generations[0][0];
        tokenTracker.
updateTokens(message.usage_metadata);
        console.log('Current Token Usage:', tokenTracker.getCurrentUsage());
        console.log('Token Differences:', tokenTracker.getTokenDiff());
      },
    },
  ],
});
```

Imports the TokenTracker class from the TokenTracker.js file to manage token usage

Callback function triggered when an LLM call completes

Updates the token tracker with usage metadata from the message

Logs the current token usage statistics to the console